If you have a high-volume site, keeping the size of the session data small and going with `cookie_store` is the way to go.

If we rule out memory store as being too simplistic, file store as too restrictive, and `memcached` as overkill, the server-side choices boil down to CookieStore, Active Record store, and DRb-based storage. Should you need to store more in a session than you can with cookies, we recommend you start with an Active Record solution. If, as your application grows, you find this becoming a bottleneck, you can migrate to a DRb-based solution.

Session Expiry and Cleanup

One problem with all the server-side session storage solutions is that each new session adds something to the session store. This means you'll eventually need to do some housekeeping or you'll run out of server resources.

Another reason to tidy up sessions is that many applications don't want a session to last forever. Once a user has logged in from a particular browser, the application might want to enforce a rule that the user stays logged in only as long as they're active; when they log out or some fixed time after they last use the application, their session should be terminated.

You can sometimes achieve this effect by expiring the cookie holding the session ID. But this is open to end-user abuse. Worse, it's hard to synchronize the expiry of a cookie on the browser with the tidying up of the session data on the server.

We therefore suggest you expire sessions by simply removing their server-side session data. Should a browser request subsequently arrive containing a session ID for data that's been deleted, the application will receive no session data; the session will effectively not be there.

Implementing this expiration depends on the storage mechanism being used.

For Active Record--based session storage, use the `updated_at` columns in the `sessions` table. You can delete all sessions that have not been modified in the last hour (ignoring daylight saving time changes) by having your sweeper task issue